PUBLIC

Reserve Governor — MIESC audit triage

Target: reserve-protocol/reserve-governor @ dc27a684

Scope: Cantina competition — first-party contracts/ (31 files, ~2.7k LoC)

Scan: 0 Critical · 0 High · 9 Medium · 19 Low · 38 Info

Verdict: the static scanner is a clean negative (0 High), matching the competition's judged 0 valid Highs; the real risk surface is governance / trust-boundary logic with no detector class

MIESC — smart-contract security audit tooling · dogfooding case study. Triage layer over a default static scan. Public information from public repositories only; static analysis only; no exploitation; nothing submitted to any program.

Reserve Governor — MIESC audit triage memo

Target: `reserve-protocol/reserve-governor` @ `dc27a684` · Cantina competition scope (first-party `contracts/`, 31 files / ~2.7k LoC) **What it is: Reserve Governor**, a hybrid optimistic/standard on-chain governance system for the Reserve protocol, built on OpenZeppelin Governor + TimelockController. Five runtime components: a `StakingVault` (ERC4626 vote-locking vault with dual delegation, multi-token rewards and an unstaking delay), its `UnstakingManager` (time-locked withdrawals), `ReserveOptimisticGovernor` (unifies a *fast/optimistic* path — short veto window, no affirmative vote — and a *slow/standard* voting path in shared OZ Governor storage), `OptimisticSelectorRegistry` (a whitelist of allowed `(target, selector)` pairs the fast path may call), and `TimelockControllerOptimistic` (a single timelock with a bypass for the optimistic path). A fast proposal that accrues enough AGAINST votes during its veto window auto-spawns a full confirmation vote; a per-account throttle limits optimistic proposals per 24h. **Toolchain:** Foundry / **solc 0.8.33** (evm *cancun*, optimizer 94, no via-IR) + **Slither 0.11.3**, reusing forge build-info. Dependencies are npm (`@openzeppelin/contracts{,-upgradeable}@5.4.0` with the project's pnpm patch making two OZ storage accessors `internal`, `@prb/math`), staged into `node_modules/` without running the project's install scripts. **Scan (contest scope = `contracts/`): 66 owned findings — 0 Critical / 0 High / 9 Medium / 19 Low / 38 Info**. **Method note:** MIESC ran Slither with its **default detector set**, deliberately *without* the project's LightChaser config, over the whole first-party `contracts/`. 368 dependency results under `node_modules/` were excluded — including **4 `node_modules/` Highs**: 3 `arbitrary-send-eth` inside OZ's own `TimelockControllerUpgradeable._execute`, `GovernorUpgradeable._executeOperations` and `GovernorUpgradeable.relay` (the timelock/governor's **by-design** arbitrary `.call{value}` executing a governance-approved target — the detector correctly identifying a send that *is the feature*), and 1 `incorrect-exp` in OZ `Math.sol`. **None is a first-party finding.**

> Triage layer. **Nothing here is a submission.** No PoC was built or submitted. The Cantina competition on this commit is **complete** (team-judged; the public leaderboard shows the outcome), so this write-up carries no live-competition embargo; it reconciles MIESC's output against the public leaderboard and the project's **own published pre-contest security review** (committed in the repo). Any disclosure is the user's own action.

Headline

On this target the static scanner is a clean negative: 0 Critical, 0 High — and that matches the competition's judged outcome exactly. Reserve Governor is governance code — voting paths, a veto window, a selector whitelist, a timelock with an optimistic bypass, an ERC4626 staking vault — and across the whole first-party `contracts/` set MIESC's default Slither run produced **not a single High**. The Cantina competition closed the same way: the public leaderboard shows **every ranked submission scoring 0 in the High column** (the pot paid the Medium tier, not the High tier) — **0 valid High findings** out of 1,465 submissions. The scanner's 9 owned Mediums are all benign detector noise (`incorrect-equality`, an ERC4626 `_withdraw` `reentrancy-no-eth`, ignored `EnumerableSet` / `deposit` return values). **Ninth target in a row with 0 exploitable Highs from the static scan** — and the one where the lens is quietest, because the entire real risk surface here is **governance-logic and trust-boundary invariants** that no pattern detector has a vocabulary for. This is the mirror image of a vault/lending target like Doppler or Royco, where the scanner is *loud* and triage clears the noise: here it is *silent*, and the value is entirely in the semantic review the noise would otherwise distract from.

No High to triage — what the scanner did produce

There is no High-severity site to open: the owned scan is **0 Critical / 0 High**. The 9 owned Mediums, characterized:

Detector	Count	Site(s)	Verdict
`unused-return`	6	`OptimisticSelectorRegistry` (`_add` / `_remove` / `initialize`), `StakingVault` (`_authorizeUpgrade` , `_moveOptimisticDelegateVotes`), `UnstakingManager.cancelLock`	Ignored `EnumerableSet.add/remove` booleans and the ignored `vault.deposit` share return — standard; no state-correctness impact. The `cancelLock` hit is the ignored `vault.deposit` return, **not** a value path (see below).
`incorrect-equality`	2	`StakingVault._calculateHandout` , `._moveOptimisticDelegateVotes`	`== 0` is intended in reward-handout and delegate-move math; intended.
`reentrancy-no-eth`	1	`StakingVault._withdraw`	Genuinely not CEI-ordered: the external call is `unstakingManager.createLock(...)` (L287) and `nativeBalanceLastKnown` (a reward-accounting balance cache) is written after it (L292). Benign not by ordering but by **callee trust** — `UnstakingManager` is created by the vault itself (`new UnstakingManager(...)` , L176), protocol-internal, so it cannot reenter adversarially.

Low/Info (57) are the usual `naming-convention` , `too-many-digits` , `timestamp` , `calls-loop` , `assembly` , `dead-code` , `pragma` / `solc-version` classes — characterized by detector class, not individually opened.

Where the static lens ends — Reserve Governor's real residual risk

Everything the competition and the project's own review actually found is **governance logic**, a class with no Slither detector. From the repo's committed **pre-contest security review** (2026-02-27) — a *known-issues* baseline the competition listed as out-of-scope, so these are issues already fixed or accepted by `dc27a684` , not the contest's judged findings:

- **[M-01] CEI-ordering fund-theft in `UnstakingManager.cancelLock`** — the review found `cancelLock` ran `forceApprove` / `vault.deposit` *before* `delete locks[lockId]` , so a callback during the external call could re-cancel and drain other users' locks. **This is fixed at the contest commit**: `cancelLock` now deletes the lock (L64) *before* the `forceApprove` / `deposit` (L66-67) — CEI-correct. Slither's flag on the same function is an unrelated benign `unused-return` on the ignored `vault.deposit` share count, not a reentrancy flag; the review itself notes the original bug needed a token with transfer callbacks (ERC-777-style) to be exploitable — a semantic precondition, like Royco's, that no detector models.
- **[M-02] reward accounting uses current balance** — `_currentAccountedNativeRewards` priced rewards off the live balance; an accounting-correctness issue. Also addressed at this commit: it now derives rewards from the cached `nativeBalanceLastKnown - totalDeposited` (L245-250), not `address(this)` 's live balance.
- **Lows in the optimistic machinery** — EOA-check bypass (`[L-01]`), optimistic/standard proposals colliding in a shared ID space (`[L-08]`), optimistic proposal state boundaries wrong due to a `>=` comparison (`[L-11]`), `ThrottleLib` rounding reducing proposal capacity (`[L-02]`), `ExecuteBatchBypass` restriction (`[L-04]`), selector-registry zero-governor (`[L-10]`).

Every one of these is a **business-logic or trust-boundary invariant** — "the fast path may only call whitelisted `(target, selector)` pairs," "optimistic and standard proposals must not share IDs," "a veto threshold must spawn exactly one confirmation vote," "the timelock bypass applies only to the optimistic path." These are precisely the properties a pattern detector cannot encode: there is no `arbitrary-send` , `reentrancy` or `uninitialized` shape to match. The scanner's correct response to this code is **silence at High severity** — which is exactly what it produced. The discrimination that matters here (is the selector whitelist complete? can an optimistic proposal escape its veto? does the throttle actually bound proposal creation?) is semantic review, start to finish.

Reconciliation against the public record

Two distinct baselines:

1. **Cantina competition — judged outcome (public leaderboard).** 1,465 submissions; the leaderboard shows **0 valid Highs** (every ranked entrant's High column is 0) and the prize settling at the **Medium tier** ($10k trigger), i.e. valid **Mediums only**. MIESC's **0-High** scan agrees with the judged High outcome. The Medium finding *titles* are gated behind Cantina login, so they are not reproduced here; what is public is that they exist and that no High was validated.
2. **Pre-contest security review (2026-02-27), committed in the repo.** Known-issues baseline, out-of-scope by the competition's own rules. **0 High / 0 Critical**, 2 Mediums (the `cancelLock` CEI bug — fixed before this commit — and the reward-balance accounting nit) plus Lows, all in the governance/vault logic. None is a Slither class.

MIESC's static scan surfaced **zero** Highs and its Mediums are benign detector noise — concordant with both baselines, and a **true negative** on the High tier rather than a conflict. The static layer added no false alarm at High severity and, correctly, no signal either: on governance code the real work is semantic.

Bottom line

- **No submittable bug** from static scan + triage — **ninth** target in a row (Lombard, Doppler, Valantis STEX, YieldNest, Arcadia, v4lend, Malda, Royco Dawn, **Reserve Governor**), all with 0 exploitable scanner Highs.
- **This target produced 0 scanner Highs at all** — and that is the correct result, matching the competition's judged **0 valid Highs** out of 1,465 submissions. The 9 owned Mediums are benign detector noise (`incorrect-equality` , ERC4626 `_withdraw` reentrancy, ignored `EnumerableSet` / `deposit` returns).
- **The real residual risk is entirely out of the static lens:** CEI ordering (the `cancelLock` M-01 — already fixed before the contest commit), reward accounting, optimistic/standard proposal ID collisions, veto-threshold logic, selector-registry whitelisting, timelock-bypass correctness, throttle rounding. These are governance / trust-boundary invariants with no detector class — the opposite end of the spectrum from a vault/lending target, and the clearest case in this directory of a scan whose honest contribution is a **clean negative**.
- **Scoping point:** the only `arbitrary-send-eth` Highs in the entire run sit inside OZ's own `Governor` / `Timelock` base contracts (the timelock/governor's by-design arbitrary execution — `_execute` , `_executeOperations` , `relay`), correctly excluded as dependencies — a scanner that counted `node_modules/` as first-party would manufacture 3 phantom "governance" Highs.
- **Honest limits:** this is the **public contest commit** after **1,465 submissions** plus a prior professional review, so anything the scanner surfaces is already known; verdicts are from reading the code against the NatSpec, the public leaderboard and the committed review; **no PoC was built or run**. Contest Medium titles are gated and not reproduced.

Appendix — Evidence trail

- Normalized scan: `results.json` (owned = first-party `contracts/` ; `node_modules/` dependencies excluded — the 4 excluded dep Highs are 3 `arbitrary-send-eth` in OZ base execution paths — `TimelockControllerUpgradeable._execute` , `GovernorUpgradeable._executeOperations` , `GovernorUpgradeable.relay` — and 1 `incorrect-exp` in OZ `Math` , no first-party `arbitrary-send`).
- Reports: `triage-focused.pdf` (focused triage); the full premium PDF is regenerable from `results.json` and is not committed.
- Toolchain: HTTPS clone @ `dc27a684` ; npm deps (`@openzeppelin/contracts{,-upgradeable}@5.4.0` + the repo's pnpm patch promoting `_getGovernorStorage` / `_getTimelockControllerStorage` to `internal` , `@prb/math@4.1.0` , `forge-std v1.14.0`) staged into `node_modules/` via `npm pack` + extract, without running the project's install scripts → `forge build --root <repo> --build-info --skip test/script` (solc 0.8.33, evm cancun) → `slither <repo> --foundry-ignore-compile --json` (Slither 0.11.3, default detectors). Build/scan were invoked from a safe working directory with the repo passed by absolute path (never

`cd`-ing into the untrusted clone); Slither's `filename_relative` paths carry a `reserve-audit/repo/` prefix, which the normalizer strips before scope-matching.

- Target pin: `reserve-protocol/reserve-governor` @ `dc27a68463cec356ff18bbdd3d8edfe9b2534372` (Cantina "Reserve Governor" competition, 30 Apr–10 May 2026).
- Only **public source from the public GitHub repository**, the **public Cantina leaderboard**, and the project's **own committed security review** were accessed; **static analysis only**, no live systems / RPC / credentials, no exploitation, nothing submitted anywhere.